

Exerciții noi — vectori, stil LeetCode

Regulile jocului, ca până acum: datele se citesc din `data.txt` (prima linie `n`, a doua linie cele `n` numere; unele exerciții au parametri în plus — scrie la fiecare). Pentru fiecare exercițiu scrii **funcția cu semnătura dată** în `functii.h` și o funcție `solutieN()` în `solutii.h` care citește, apelează și afișează. Nu modifica semnăturile.

Dificultate: ★ ușor · ★★ mediu · ★★★ provocare.

1. Perechi cu suma dată ★

LeetCode-ul original: Two Sum.

Se dă un vector și un număr `S` (a treia linie din `data.txt`). Găsește **prima pereche de poziții** `(i, j)` cu `i < j` și `v[i] + v[j] == S`. Dacă nu există, afișează `-1 -1`.

```
void perecheSuma(int v[], int n, int S, int &pi, int &pj);
```

Exemplu — `data.txt`:

```
5
2 7 11 15 3
9
```

Output: `0 1` (pentru că $2 + 7 = 9$)

Verifică-te: ce afișează pentru `S = 100`? Dar dacă `S` e dublul unui element care apare o singură dată — ai grijă să nu folosești același element de două ori.

2. Cel mai bun moment de cumpărare ★

LeetCode-ul original: Best Time to Buy and Sell Stock.

Vectorul reprezintă prețul unei acțiuni în zile consecutive. Cumperi într-o zi și vinzi într-o zi **ulterioară**. Care e profitul maxim posibil? Dacă orice tranzacție iese în pierdere, profitul e `0`.

```
int profitMaxim(int v[], int n);
```

Exemplu — data.txt:

```
6
7 1 5 3 6 4
```

Output: 5 (cumperi la 1, vinzi la 6)

Verifică-te: vector strict descrescător 9 7 4 2 → trebuie să dea 0. Un singur element → 0.

Restricție: o singură parcurgere a vectorului. Fără două bucle imbricate.

3. Mută zerourile la final ★

LeetCode-ul original: Move Zeroes.

Mută toate zerourile la sfârșitul vectorului, **păstrând ordinea** celorlalte elemente. Modifici vectorul pe loc, fără un al doilea vector.

```
void mutaZerouri(int v[], int n);
```

Exemplu — data.txt:

```
5
0 1 0 3 12
```

Output: 1 3 12 0 0

Verifică-te: vector numai cu zerouri; vector fără niciun zero; zero pe ultima poziție.

4. Cea mai lungă secvență de 1 ★

LeetCode-ul original: Max Consecutive Ones.

Vectorul conține doar 0 și 1. Care e lungimea celei mai lungi secvențe de 1 consecutivi?

```
int maxUnuConsecutiv(int v[], int n);
```

Exemplu — data.txt:

```
6
1 1 0 1 1 1
```

Output: 3

Verifică-te: vector numai cu 0 → 0. Secvența cea mai lungă e chiar la finalul vectorului — o prinzi?

5. Elimină duplicatele din vector sortat ★★

LeetCode-ul original: Remove Duplicates from Sorted Array.

Vectorul e **sortat crescător**. Elimină duplicatele pe loc și returnează noua lungime. Primele `k` poziții trebuie să conțină valorile distincte, în ordine.

```
int stergeDuplicate(int v[], int n);
```

Exemplu — data.txt:

```
10
1 1 2 3 3 3 5 5 7 7
```

Output: 5 și vectorul 1 2 3 5 7

Verifică-te: toate elementele egale → lungime 1. Niciun duplicat → vectorul neatins.

Restricție: fără vector auxiliar, o singură parcurgere.

6. Elementul majoritar ★★

LeetCode-ul original: Majority Element.

Găsește elementul care apare de **mai mult de $n/2$ ori**. Se garantează că există.

```
int elementMajoritar(int v[], int n);
```

Exemplu — data.txt:

```
7
2 2 1 1 1 2 2
```

Output: 2

Verifică-te: vector cu un singur element; vector în care majoritarul e pe toate pozițiile.

Provocare (după ce merge): ai rezolvat cu vector de frecvență sau cu două bucle? Există o soluție cu o singură parcurgere și **fără** vector de frecvență. Nu o căuta pe net — încearcă întâi să o găsești singur.

7. Rotire cu k poziții ★★

LeetCode-ul original: Rotate Array.

Rotește vectorul spre dreapta cu `k` poziții (`k` pe a treia linie din `data.txt`). `k` poate fi mai mare decât `n`.

```
void rotesteDreapta(int v[], int n, int k);
```

Exemplu — `data.txt`:

```
7
1 2 3 4 5 6 7
3
```

Output: 5 6 7 1 2 3 4

Verifică-te: `k = 0`, `k = n` și `k = n + 3` — primele două lasă vectorul neschimbat, al treilea e totuna cu `k = 3`.

8. Suma pe interval, de multe ori ★★

LeetCode-ul original: Range Sum Query.

Se dau `q` întrebări (a treia linie: `q`, apoi `q` linii cu câte două poziții `st dr`). Pentru fiecare, afișează suma elementelor dintre pozițiile `st` și `dr` inclusiv.

```
void construiestePrefix(int v[], int n, long long p[]);  
long long sumaInterval(long long p[], int st, int dr);
```

Exemplu — data.txt :

```
5  
1 2 3 4 5  
3  
0 2  
1 3  
0 4
```

Output:

```
6  
9  
15
```

Restricție: fiecare întrebare trebuie să coste **o singură scădere**, nu o buclă. De asta există funcția `construiestePrefix` — gândește ce trebuie să conțină `p`.

9. Produsul fără elementul curent ★★★

LeetCode-ul original: Product of Array Except Self.

Pentru fiecare poziție `i`, calculează produsul **tuturor** celorlalte elemente. **Fără împărțire** — asta e toată frumusețea problemei.

```
void produsFaraSine(int v[], int n, long long rez[]);
```

Exemplu — data.txt :

```
4  
1 2 3 4
```

Output: 24 12 8 6

Verifică-te: ce se întâmplă dacă vectorul conține un 0? Dar două zerouri? Soluția corectă le tratează fără niciun `if` special.

10. Apa dintre blocuri ★★★

LeetCode-ul original: Trapping Rain Water.

Vectorul reprezintă înălțimi de blocuri lipite. După ploaie, apa se strânge în golurile dintre blocuri. Câte unități de apă rămân?

```
int apaAdunata(int v[], int n);
```

Exemplu — `data.txt`:

```
12
0 1 0 2 1 0 1 3 2 1 2 1
```

Output: 6

Verifică-te: vector crescător → 0 (apa curge). Vector în formă de V (5 0 5) → 5 . Desenează pe hârtie exemplul mare **înainte** să scrii cod — la problema asta desenul e jumătate din soluție.

Hint de structură (nu de rezolvare): pentru fiecare poziție, apa de deasupra ei depinde de două lucruri. Care?

Ordinea recomandată

1 → 2 → 3 → 4 (încălzire, câte una pe zi) · 5 → 6 → 7 (aici începe gimnastica de indici) · 8 (te învață prefixele — instrument, nu doar problemă) · 9 → 10 (după ce restul merg fără ajutor).

Câte un commit per exercițiu, cu mesaj pe formatul `feat: exercitiul N - <nume>` . Nu trece la următorul până cel curent nu trece **toate** punctele de la „Verifică-te”.